Facultad de Ingeniería Electrónica
Programa de Ingeniería Electrónica
NLP y LLMs

Universidad Santo Tomás

Guía de Laboratorio N° 7
Comprensión de Órdenes en Español para Robots con LLMs Ligeros
6 de octubre de 2026

---

1 Información General

- **Asignatura:** NLP y LLMs (Natural Language Processing and Large Language Models)
- **Duración:** 2 semanas (trabajo autónomo + sesiones de seguimiento)
- **Modalidad:** Trabajo en equipos de 2 a 3 estudiantes
- **Herramientas:** Python 3.x, Ollama o llama.cpp, Hugging Face Transformers, spaCy
- **Componentes de voz:** Whisper (whisper.cpp o faster-whisper), Vosk, Piper
- **Entregable:** Repositorio con sistema funcional, informe de 4 páginas y demostración en vivo

2 Objetivos

1. Construir un sistema completo de comprensión de órdenes en español que integre voz a texto (S2T), un LLM ligero y texto a voz (T2S).
2. Extraer acciones, personas, objetos, adjetivos y lugares de órdenes habladas, devolviendo un JSON estructurado.
3. Comparar al menos tres modelos de lenguaje de menos de 4 mil millones de parámetros en términos de acierto, latencia y memoria.
4. Evaluar el impacto de la transcripción (S2T) en el rendimiento final del sistema.
5. Reflexionar sobre el equilibrio entre precisión y costo computacional en sistemas de IA embebida.

3 Competencias a Desarrollar

- Diseño de pipelines de NLP de extremo a extremo con componentes abiertos.
- Habilidad para comparar modelos de lenguaje ligeros bajo restricciones de hardware.
- Destreza en la integración de componentes de voz (S2T y T2S) en CPU.
- Análisis crítico del trade-off entre acierto, latencia y memoria.
- Trabajo colaborativo en equipos de investigación aplicada.

4 Marco Teórico

4.1 Comprensión de Órdenes en Robots

La comprensión de órdenes en lenguaje natural es una tarea fundamental en robótica de servicio. El robot debe identificar:

- **Acciones:** Lo que debe hacer, en infinitivo (llevar, traer, buscar, entregar).
- **Personas:** Alguien nombrado o descrito (María, el señor alto).
- **Objetos:** Cosas que se manipulan o buscan (taza, libro, paquete).
- **Adjetivos:** Rasgos que describen a personas, objetos o lugares (roja, grande, alto).
- **Lugares:** Origen, destino o ubicación (cocina, sala, oficina).

4.2 El Formato JSON de Salida

El sistema debe devolver un único objeto JSON válido, sin texto adicional. Este formato fijo permite calificar de forma automática y conectar el sistema al robot sin adaptaciones. Ejemplo:

```json
{
  "transcripcion": "Lleva la taza roja a la cocina y entragasela a Maria",
  "acciones": [
    {
      "accion": "llevar",
      "objetos": [{"nombre": "taza", "adjetivos": ["roja"]}],
      "personas": [],
      "lugares": [{"nombre": "cocina", "adjetivos": []}]
    },
    {
      "accion": "entregar",
      "objetos": [{"nombre": "taza", "adjetivos": ["roja"]}],
      "personas": [{"nombre": "Maria", "adjetivos": []}],
      "lugares": []
    }
  ],
  "respuesta": "Voy a llevar la taza roja a la cocina y a entregarsela a Maria"
}
```

Listing 1: JSON de salida para una orden compuesta.

4.3 Los Cinco Niveles de Dificultad

El conjunto de datos cubre cinco niveles, en proporciones similares entre escenarios de hogar y oficina de recepción:

Nivel	Qué evalúa	Ejemplo en hogar	Ejemplo en recepción
1	Simple	"Trae el vaso"	"Saluda al visitante"
2	Descriptiva	"Pon el libro azul sobre la mesa pequeña"	"Lleva el paquete grande a la oficina principal"
3	Compuesta	"Ve al cuarto, toma la cobija gris y trácesela a la abuela"	"Recibe a la señora Gómez, acompañála a la sala de reuniones y avísale a Carlos"
4	Coloquial	"oye porfa me alcanzas el celular que dejé en el cuarto"	"eh mira dile al señor de la corbata roja que ya lo atienden"
5	Trampa	"Tráeme eso"	"Qué demora la de hoy"

Table 1: Niveles de dificultad del conjunto de datos.

4.4 Reglas de Anotación

1. La acción va en infinitivo y en minúscula ("tráeme" se anota "traer").
2. Los nombres van como aparecen en la orden, sin artículo y en singular o plural según el original.
3. Los adjetivos van como aparecen ("roja", no "rojo").
4. Los pronombres se reemplazan por la entidad a la que se refieren. Si no hay referente ("tráeme eso"), no se anota nada.
5. "Tú" (el robot) y "yo" (quien habla) no se anotan como personas, salvo que la orden dé un nombre.
6. Una orden que no es una instrucción para el robot devuelve {`"acciones": []`}.

4.5 Restricciones Computacionales

Restricción	Límite propuesto
Tamaño del modelo de lenguaje	Hasta 4 mil millones de parámetros
Memoria RAM en ejecución	Hasta 8 GB para el sistema completo
Hardware de evaluación	Solo CPU, sin GPU
Conexión a Internet	Ninguna durante la inferencia
Licencia	Debe permitir uso académico y redistribución

Table 2: Restricciones del sistema completo.

5 Materiales y Recursos

- Computador con Python 3.x y al menos 8 GB de RAM.
- 150 órdenes de desarrollo (audio, transcripción correcta y JSON anotado).
- Script de evaluación proporcionado por el organizador.
- Modelos sugeridos: Gemma, Qwen, Phi, Llama (variantes pequeñas), SmolLM.
- Herramientas de ejecución local: Ollama, llama.cpp (formato GGUF).
- Componentes de voz: Whisper (whisper.cpp o faster-whisper), Vosk, Piper.
- spaCy en español para lematización y verificación gramatical.

6 Procedimiento Experimental (2 semanas)

Organización del Tiempo

Días	Actividad	Entrega
1 a 3	Lanzamiento, entrega de datos y script. Armar cadena completa con componentes sin ajustar.	Línea base de extremo a extremo
4 a 7	Comparación de modelos de lenguaje y opciones de voz a texto.	30 órdenes grabadas y anotadas por equipo
8 a 11	Optimización: prompts, salida restringida, cuantización. Tabla de posiciones parcial el día 10.	Envío parcial opcional
12 a 14	Cierre, evaluación en máquina de referencia y demostraciones.	Repositorio, informe y demo

Table 3: Cronograma sugerido para las 2 semanas.

6.1 Parte 1: Línea Base del Sistema (Días 1–3)

Paso 1: Configuración del entorno

Instalar las dependencias necesarias.

```
!pip install transformers torch soundfile
!pip install faster-whisper  # o whisper.cpp
!pip install piper-tts
!pip install spacy
!python -m spacy download es_core_news_sm
!pip install ollama  # o descargar llama.cpp
```

Listing 2: Instalación de dependencias.

Paso 2: Voz a texto (S2T)

Implementar la transcripción de audio a texto con Whisper o Vosk.

```
from faster_whisper import WhisperModel

model_size = "small"  # Variantes: tiny, base, small, medium
model = WhisperModel(model_size, device="cpu", compute_type="int8")

def transcribir(audio_path):
    segments, info = model.transcribe(audio_path, language="es")
    transcripcion = " ".join([seg.text for seg in segments])
    return transcripcion.strip()

# Ejemplo
texto = transcribir("audios/orden_001.wav")
print(texto)
```

Listing 3: Transcripción con faster-whisper.

Paso 3: Comprensión con LLM ligero

Cargar un modelo ligero y diseñar un prompt para extraer el JSON.

```
PROMPT_SISTEMA = """Eres un asistente que extrae informacion de ordenes en espanol.
Devuelve UNICAMENTE un JSON valido con esta estructura:
{
  "acciones": [
    {
      "accion": "infinitivo",
      "objetos": [{"nombre": "...", "adjetivos": ["..."]}],
      "personas": [{"nombre": "...", "adjetivos": ["..."]}],
      "lugares": [{"nombre": "...", "adjetivos": ["..."]}]
    }
  ]
}

Reglas:
1. La accion va en infinitivo y minuscula.
2. Los nombres van sin articulo.
3. Los adjetivos van como aparecen.
4. Los pronombres se reemplazan por su referente.
5. "tu" y "yo" no se anotan como personas.
6. Si no es una orden, devuelve {"acciones": []}.
7. NO inventes entidades. Si falta informacion, deja listas vacias.
"""

def extraer_json(transcripcion, modelo="qwen2.5:1.5b"):
    import ollama
    respuesta = ollama.chat(
        model=modelo,
        messages=[
            {"role": "system", "content": PROMPT_SISTEMA},
            {"role": "user", "content": f"Orden: {transcripcion}"}
        ],
        format="json"  # Salida restringida a JSON
    )
    return respuesta["message"]["content"]
```

Listing 4: Prompt para extracción de entidades.

Paso 4: Texto a voz (T2S)

Generar la respuesta hablada a partir del JSON usando plantillas.

```
import json
import subprocess

def generar_respuesta(json_str):
    data = json.loads(json_str)
    acciones = data.get("acciones", [])

    if not acciones:
        return "No entendi ninguna orden."

    # Verificar si falta informacion
    for acc in acciones:
        if not acc["objetos"] and not acc["personas"]:
            return " Que  quieres que te traiga?"

    # Confirmacion simple con plantilla
    partes = []
    for acc in acciones:
        obj = acc["objetos"][0]["nombre"] if acc["objetos"] else ""
        adj = " ".join(acc["objetos"][0]["adjetivos"]) if acc["objetos"] else ""
```

```
        lugar = acc["lugares"][0]["nombre"] if acc["lugares"] else ""
        persona = acc["personas"][0]["nombre"] if acc["personas"] else ""
        partes.append(f"Voy a {acc['accion']} {adj} {obj} {lugar} {persona}".strip())

    return " y ".join(partes)

def sintetizar(texto, salida="respuesta.wav"):
    subprocess.run([
        "piper", "--model", "es_ES-davefx-medium.onnx",
        "--output_file", salida
    ], input=texto.encode())
    return salida
```

Listing 5: Generación de respuesta y síntesis con Piper.

Paso 5: Pipeline completo

Integrar los tres componentes en un solo script.

```
import json

def pipeline(audio_path):
    # 1. S2T
    transcripcion = transcribir(audio_path)

    # 2. Comprension
    json_str = extraer_json(transcripcion)
    data = json.loads(json_str)

    # 3. Respuesta
    respuesta = generar_respuesta(json_str)
    data["transcripcion"] = transcripcion
    data["respuesta"] = respuesta

    # 4. T2S
    audio_respuesta = sintetizar(respuesta)

    return json.dumps(data, ensure_ascii=False, indent=2)

# Ejemplo
print(pipeline("audios/orden_001.wav"))
```

Listing 6: Pipeline de extremo a extremo.

6.2 Parte 2: Comparación de Modelos (Días 4–7)

Paso 1: Selección de modelos

Probar al menos tres modelos de lenguaje de menos de 4B parámetros:

Familia	Organización	Por qué probarla
Gemma	Google	Variantes muy pequeñas con buen soporte multilingüe
Qwen	Alibaba	Amplia gama de tamaños, desde menos de mil millones
Phi	Microsoft	Diseñada para razonar bien con pocos parámetros
Llama (pequeñas)	Meta	Ecosistema amplio y versiones cuantizadas
SmolLM	Hugging Face	Modelos diminutos para medir el piso de tamaño útil

Table 4: Modelos sugeridos para comparación.

Paso 2: Métricas de evaluación

Medir para cada modelo:

- **F1 por categoría:** promedio de las cinco categorías (acción, persona, objeto, adjetivo, lugar).
- **Estructura:** porcentaje de entidades y adjetivos asignados correctamente.
- **Robustez:** F1 solo en niveles 4 y 5.
- **JSON válido:** porcentaje de salidas que cumplen el esquema.
- **Latencia:** mediana de segundos por orden (de fin de audio a inicio de respuesta).
- **Memoria:** pico de RAM del sistema completo.

```python
import time
import psutil
import json
from sklearn.metrics import f1_score

def evaluar_modelo(modelo, audios, anotaciones):
    resultados = []
    for audio, anotacion in zip(audios, anotaciones):
        # Medir latencia
        inicio = time.time()
        pred = pipeline_con_modelo(audio, modelo)
        latencia = time.time() - inicio

        # Medir memoria
        memoria = psutil.Process().memory_info().rss / 1e9  # GB

        # Comparar con anotacion
        pred_json = json.loads(pred)
        f1 = calcular_f1(pred_json, anotacion)

        resultados.append({
            "latencia": latencia,
            "memoria": memoria,
            "f1": f1,
            "json_valido": es_json_valido(pred)
        })

```

```
    return resultados
```

Listing 7: Script de evaluación de métricas.

Paso 3: Tabla comparativa

Construir una tabla como la siguiente:

Modelo	F1	Latencia	RAM	JSON	Tamaño
Gemma-2B	0.82	1.8s	2.5 GB	98%	1.5 GB
Qwen2.5-1.5B	0.79	1.2s	1.8 GB	99%	1.0 GB
Phi-3-mini	0.85	2.1s	3.2 GB	97%	2.2 GB
SmolLM-1.7B	0.74	0.9s	1.2 GB	96%	1.1 GB

Table 5: Ejemplo de tabla comparativa de modelos.

6.3 Parte 3: Optimización (Días 8–11)

Paso 1: Salida restringida a JSON

Forzar la salida del LLM a un esquema JSON para eliminar salidas inválidas.

```
respuesta = ollama.chat(
    model="qwen2.5:1.5b",
    messages=[...],
    format="json"  # Fuerza JSON valido
)
```

Listing 8: Salida restringida con Ollama.

Paso 2: Cuantización

Usar modelos cuantizados en formato GGUF para reducir memoria y latencia.

```
# Descargar modelo cuantizado
# ollama pull qwen2.5:1.5b-q4_K_M

# O usar llama.cpp directamente
# ./main -m modelos/qwen2.5-1.5b-q4_K_M.gguf -p "Orden: trae el vaso"
```

Listing 9: Ejecución con llama.cpp y cuantización Q4.

Paso 3: Prompts con ejemplos

Incluir ejemplos en el prompt (few-shot) para mejorar el rendimiento en niveles difíciles.

```
PROMPT_FEWSHOT = """Extrae el JSON de las siguientes ordenes.

Ejemplo 1:
Orden: "Trae el vaso"
Salida: {"acciones": [{"accion": "traer", "objetos": [{"nombre": "vaso", "adjetivos":
    []}], "personas": [], "lugares": []}]}

Ejemplo 2:
Orden: "Lleva la taza roja a la cocina y entragasela a Maria"
Salida: {"acciones": [{"accion": "llevar", "objetos": [{"nombre": "taza", "adjetivos": [
    "roja"]}], "personas": [], "lugares": [{"nombre": "cocina", "adjetivos": []}]}, {"
    accion": "entregar", "objetos": [{"nombre": "taza", "adjetivos": ["roja"]}], "
    personas": [{"nombre": "Maria", "adjetivos": []}], "lugares": []}]}
```

```
10
11 Ejemplo 3:
12 Orden: "Que demora la de hoy"
13 Salida: {"acciones": []}
14
15 Ahora procesa:
16 Orden: "{orden}"
17 Salida:
18 """
```

Listing 10: Prompt con ejemplos few-shot.

Paso 4: Apoyo con spaCy

Usar spaCy para lematizar verbos y verificar categorías gramaticales.

```python
import spacy

nlp = spacy.load("es_core_news_sm")

def lematizar_verbo(verbo):
    doc = nlp(verbo)
    for token in doc:
        if token.pos_ == "VERB":
            return token.lemma_
    return verbo

# Ejemplo
print(lematizar_verbo("traeme"))  # "traer"
print(lematizar_verbo("entregasela"))  # "entregar"
```

Listing 11: Lematización con spaCy.

6.4 Parte 4: Cierre y Demostración (Días 12–14)

Paso 1: Evaluación en máquina de referencia

Ejecutar el sistema completo en el equipo de referencia y medir todas las métricas.

Paso 2: Análisis de errores

Identificar los errores más frecuentes:

- Errores de transcripción (S2T): ”casa” en vez de ”taza”.
- Errores de comprensión (LLM): pronombres mal resueltos, entidades inventadas.
- Errores de formato: JSON inválido.
- Errores de respuesta: confirmación incorrecta o falta de pregunta de aclaración.

Paso 3: Demostración en vivo

Preparar una demostración de 5 minutos con órdenes dictadas por el jurado.

7 Preguntas Guía para los Equipos

Nota: Las respuestas deben ser discutidas en equipo y presentadas en el informe final, justificadas con referencias bibliográficas.

1. ¿Por qué el reto enfatiza el equilibrio entre acierto y costo computacional en lugar del modelo más grande?
2. ¿Qué ventajas ofrece la salida restringida a JSON frente a la generación libre de texto?
3. ¿Cómo afecta la cuantización al rendimiento del modelo en términos de F1, latencia y memoria?
4. ¿Qué estrategias se pueden usar para resolver pronombres como "entrégasela" sin acceso a internet?
5. ¿Cómo se puede detectar y penalizar la invención de entidades en el JSON de salida?
6. ¿Qué papel juega spaCy en la mejora del pipeline de comprensión?
7. ¿Cómo se propaga el error del S2T al LLM y cómo se puede mitigar?
8. ¿Qué diferencias esperas encontrar entre los niveles 1–3 y los niveles 4–5 en términos de F1?
9. ¿Qué técnicas de ajuste fino (LoRA) podrían mejorar el rendimiento sin exceder los límites de cómputo?
10. ¿Cómo se relaciona este reto con aplicaciones reales de robótica de servicio en el hogar y la oficina?

8 Entregables

1. **Repositorio** con el sistema y un único comando de ejecución:

```
python predecir.py --entrada audios/ --salida predicciones.json

```

   Debe incluir instrucciones de instalación y descarga del modelo.

2. **Informe de 4 páginas** con:
   - Los modelos de lenguaje evaluados (mínimo tres) y las opciones de voz a texto probadas (mínimo dos).
   - Una tabla de acierto frente a latencia y memoria para cada modelo.
   - La técnica elegida y justificación.
   - Análisis de los errores más frecuentes.
3. **Demostración de 5 minutos** en vivo, con órdenes dictadas por el jurado.
4. **30 órdenes nuevas** grabadas y anotadas por equipo (entrega al final de la primera semana).

9 Rúbrica de Evaluación

Criterio	Excelente (5)	Aceptable (3)	Deficiente (1)
Acierto por categoría (25 pts)	F1 promedio ≥ 0.85 en las cinco categorías	F1 entre 0.70 y 0.85	F1 < 0.70
Estructura (10 pts)	$> 95\%$ de entidades y adjetivos correctamente asignados	$80 - 95\%$	$< 80\%$
Robustez (10 pts)	F1 ≥ 0.75 en niveles 4 y 5	F1 entre 0.60 y 0.75	F1 < 0.60
JSON válido (5 pts)	$> 98\%$ de salidas válidas	$90 - 98\%$	$< 90\%$
Voz a texto (10 pts)	WER $< 10\%$	WER entre 10 y 20%	WER $> 20\%$
Respuesta hablada (5 pts)	Respuesta coherente y pregunta cuando falta info	Respuesta parcialmente coherente	Respuesta incorrecta
Latencia (15 pts)	$\leq$ 2s por orden	Entre 2 y 5s	$\geq$ 8s
Memoria y tamaño (5 pts)	$\leq$ 3 GB RAM	Entre 3 y 6 GB	$\geq$ 8 GB
Informe y reproducibilidad (15 pts)	Comparación completa, análisis de errores y un solo comando	Comparación parcial	Sin comparación

Table 6: Rúbrica de evaluación del reto.

10 Bibliografía Recomendada

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Radford, A., Kim, J. W., Hallacy, C., et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. *OpenAI Technical Report*.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*.
- Honnibal, M., & Montani, I. (2017). spaCy 2: Natural Language Understanding with Bloom Embeddings, Convolutional Neural Networks and Incremental Parsing. *Explosion AI*.
- Hu, E. J., Shen, Y., Wallis, P., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*.
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing*. Stanford University.
- Gemma Team, Google DeepMind. (2024). Gemma: Open Models Based on Gemini Research and Technology.

- Qwen Team, Alibaba. (2024). Qwen2.5 Technical Report.
- Microsoft. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone.

11 Anexos

11.1 A. Código completo del pipeline

```python
import json
import subprocess
from faster_whisper import WhisperModel
import ollama

# --- Configuracion ---
WHISPER_MODEL = "small"
LLM_MODEL = "qwen2.5:1.5b"
PIPER_MODEL = "es_ES-davefx-medium.onnx"

PROMPT = """Eres un asistente que extrae informacion de ordenes en espanol.
Devuelve UNICAMENTE un JSON valido con esta estructura:
{"acciones": [{"accion": "...", "objetos": [...], "personas": [...], "lugares": [...]}]}
Reglas:
1. Accion en infinitivo y minuscula.
2. Nombres sin articulo.
3. Adjetivos como aparecen.
4. Pronombres reemplazados por su referente.
5. "tu" y "yo" no se anotan como personas.
6. Si no es orden, {"acciones": []}.
7. NO inventes entidades.
"""

# --- S2T ---
def transcribir(audio_path):
    model = WhisperModel(WHISPER_MODEL, device="cpu", compute_type="int8")
    segments, _ = model.transcribe(audio_path, language="es")
    return " ".join([seg.text for seg in segments]).strip()

# --- Comprension ---
def extraer_json(transcripcion):
    respuesta = ollama.chat(
        model=LLM_MODEL,
        messages=[
            {"role": "system", "content": PROMPT},
            {"role": "user", "content": f"Orden: {transcripcion}"}
        ],
        format="json"
    )
    return respuesta["message"]["content"]

# --- Respuesta ---
def generar_respuesta(json_str):
    data = json.loads(json_str)
    acciones = data.get("acciones", [])
    if not acciones:
        return "No entendi ninguna orden."
    for acc in acciones:
        if not acc["objetos"] and not acc["personas"]:
            return " Que  quieres que te traiga?"
    partes = []
```

```
    for acc in acciones:
        obj = acc["objetos"][0]["nombre"] if acc["objetos"] else ""
        adj = " ".join(acc["objetos"][0]["adjetivos"]) if acc["objetos"] else ""
        lugar = acc["lugares"][0]["nombre"] if acc["lugares"] else ""
        persona = acc["personas"][0]["nombre"] if acc["personas"] else ""
        partes.append(f"Voy a {acc['accion']} {adj} {obj} {lugar} {persona}".strip())
    return " y ".join(partes)

# --- T2S ---
def sintetizar(texto, salida="respuesta.wav"):
    subprocess.run(
        ["piper", "--model", PIPER_MODEL, "--output_file", salida],
        input=texto.encode()
    )
    return salida

# --- Pipeline ---
def pipeline(audio_path):
    transcripcion = transcribir(audio_path)
    json_str = extraer_json(transcripcion)
    data = json.loads(json_str)
    respuesta = generar_respuesta(json_str)
    data["transcripcion"] = transcripcion
    data["respuesta"] = respuesta
    sintetizar(respuesta)
    return json.dumps(data, ensure_ascii=False, indent=2)

if __name__ == "__main__":
    print(pipeline("audios/orden_001.wav"))
```

Listing 12: Pipeline completo de comprensión de órdenes.

11.2 B. Recomendaciones para el trabajo en equipo

- Definir roles: líder de proyecto, responsable de S2T, responsable de LLM, responsable de T2S, responsable de documentación.
- Revisar conjuntamente los resultados antes de entregar.
- Documentar todas las decisiones y hallazgos en el repositorio.
- Citar correctamente todas las fuentes consultadas.
- Incluir capturas de pantalla de las tablas comparativas y análisis de errores.
- Asegurar que el sistema corra con un solo comando en la máquina de referencia.

11.3 C. Preguntas adicionales para la discusión

- ¿Qué ventajas y desventajas tiene usar un modelo entrenado principalmente en inglés para comprender órdenes en español?
- ¿Cómo podrías modificar el pipeline para manejar órdenes con tres o más acciones encadenadas?
- ¿Qué otras aplicaciones de la comprensión de órdenes en robots se te ocurren en el ámbito doméstico y de oficina?
- ¿Cómo se podría integrar este sistema con un robot real para ejecutar las acciones?

- ¿Qué implicaciones éticas tiene un robot que comprende órdenes en lenguaje natural y actúa en el hogar?